
Data Structures

BS (CS) _Fall_2024

Lab_15 Tasks



Learning Objectives:

1. Graphs

Lab Tasks

Submission Instructions

1. Create a separate **.h** and **.cpp** file for each class.
2. Now create a new folder with name *ROLLNO_SEC_LAB14* e.g. *i23XXXX_A_LAB15*(Also name your cpp file like this.)
3. Move all of your files to this newly created directory and compress it into a **.zip file**. Return the test file as well.
4. Now you have to submit this zipped file on Google Classroom.
5. If you don't follow the above-mentioned submission instruction, you will be marked **zero**.
6. Plagiarism or AI generated code in the Lab Task will result in **zero** marks in the whole category.

Task 1 : Implement a Graph Data Structure using a Node-based Approach (Adjacency list) in C++

In this lab, you will be implementing a graph data structure using a node-based implementation with an adjacency list. You will be using C++ for this lab.

To get started, create a **GraphNode** class that represents a node in the graph. This class should contain the following:

1. A data member that stores the ID of the node.
2. A data member that stores a list of the node's neighbors (i.e., nodes that are connected to this node).
3. A constructor that takes the ID of the node as an argument.
4. A method that adds a neighbor to the node.

Next, create a **Graph** class that represents the entire graph. This class should contain the following:

1. A data member that stores a list of all the nodes in the graph.
2. A method that adds a node to the graph.
3. A method that adds an edge between two nodes in the graph.
4. A method that prints out the adjacency list representation of the graph.
5. Your task is to implement the GraphNode and Graph classes as described above.

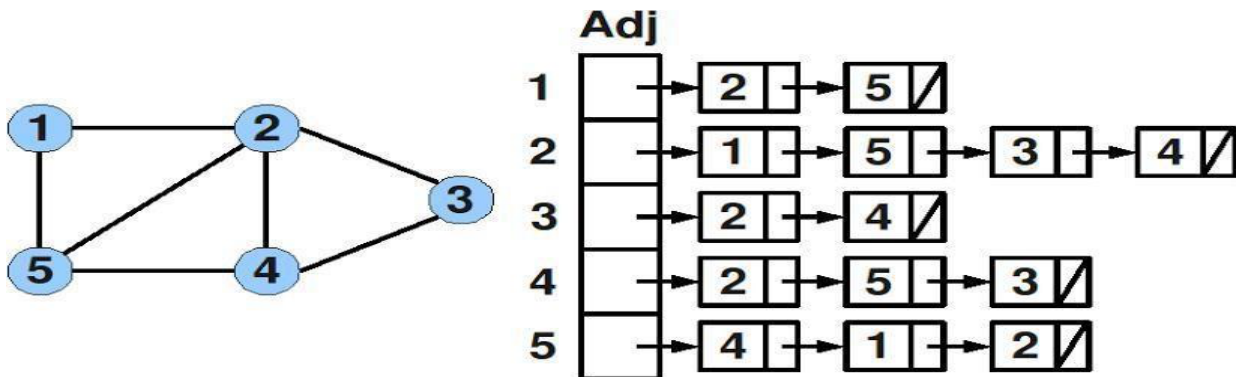
Data Items

Each vertex in a graph has a label (of type int) that uniquely identifies it. Vertices may include additional data.

Structure

The relationship between the vertices in a graph is expressed using a set of

directed/undirected Edges, where each edge connects one pair of vertices. Here is an example of undirected graph and its representation in the form of adjacency list.



Operations

Node:

```
class Node
{
public:
    int data;
    Node * next;
};
```

Adjacency List: // it's a linked list class called adjlist

```
class AdjList
{
public:
    Node *head;
    Void Insert(Node * NewNode)
    Void Display()
};
```

Graph:

```
class Graph
{
public:
```

```
int maxVertices;           //max no of vertices in your graph
int noOfVertices;         // initially 0, increments with every call to createVertex function.
                           So basically its kind of counter

AdjList *list = NULL;
```

Graph (int maxVertices)

Results:

Constructor. Creates an empty graph. Allocates enough memory for a graph containing maxNumber vertices.

~ Graph ()

Results:

Destructor. Deallocates (frees) the memory used to store a graph.

Node * createVertex (int data)

Results:

Create and return a Vertex.

void insertEdge (int src, int dest)

Requirements:

The graph includes vertices source and destination.

Results:

Inserts an undirected edge connecting vertices source and destination into a graph.

void showGraphStructure ()

Results:

Outputs a graph with the vertices in array form and the edges in adjacency list form. If the graph is empty, outputs “Empty graph”. Note that this operation is intended for testing/debugging purposes only.